PERSONAL EXPENSE TRACKER

A PROJECT REPORT

SUBMITTED BY

Roll Number	Registration Number	Student Code	Student Name
26102410028	26102000150	BWU/MCA/26/032	Sumi Bea

Of in partial fulfillment for the award of the degree

MASTER OF COMPUTER APPLICATION

IN

DEPARTMENT OF COMPUTATIONAL SCIENCE



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

ABSTRACT

The **Personal Expense Tracker** is a simple Python-based project developed to help users manage and monitor their daily expenses efficiently. In our daily lives, keeping track of expenses is important for maintaining a proper budget and avoiding unnecessary spending. This project provides an easy and convenient way to record expenses and understand personal spending habits.

The main objective of this project is to create a user-friendly system that allows users to add, view, and manage their daily expenses. Users can enter expense details such as the amount, category, date, and description. The system stores the entered information and helps users review their spending records whenever required. It can also calculate the total expenses, making it easier to understand how much money has been spent over a particular period.

The project is developed using **Python programming language**, which provides simple and effective features for implementing the required functionality. It helps demonstrate fundamental programming concepts such as variables, conditional statements, loops, functions, and data handling.

The Personal Expense Tracker reduces the effort required to maintain expense records manually and helps users become more aware of their spending patterns. This project is useful for students and individuals who want to develop better financial management habits. In the future, the system can be enhanced by adding graphical user interfaces, monthly budget planning, graphical reports, and database integration.

In conclusion, the Personal Expense Tracker is a basic yet useful application that demonstrates how Python programming can be used to solve everyday problems related to personal finance management.

TABLE OF CONTENTS

Chapter / Section	Title	Page No.
LIST OF TABLES		i
LIST OF FIGURES		ii
LIST OF ABBREVIATIONS		Iii
1. INTRODUCTION		1-2
1.1	Project Background and Motivation	1
1.2	Problem Statement	1
1.3	Proposed Solution	2
1.4	Scope of the Project	2
2. OBJECTIVES		3
2.1	Primary Objective	3
2.2	Secondary Objectives	3
2.3	Functional Objectives	3
2.4	Technical Objectives	3
2.5	User-Centric Objectives	4
3. PROJECT PLANNING		5-6
3.1	Need for Systematic Planning	5
3.2	Development Methodology (Agile Scrum)	5
3.3	Work Breakdown Structure (WBS)	5
3.4	Project Scheduling and Milestones	5
3.5	Resource & Tool Identification	6
3.6	Risk Analysis and Mitigation Strategies	6
4. REQUIREMENT ANALYSIS		7-8
4.1	User Requirements	7
4.2	Functional Requirements	7
4.3	Non-Functional Requirements	7
4.4	Hardware and Software Specifications	8
4.5	Data Requirements and Formats	8
4.6	Comparative Study of Existing Systems	8
5. SYSTEM FLOW & DESIGN		9-13
5.1	System Architecture	9
5.2	Context Level Diagram (DFD Level 0)	9

5.3	Detailed Data Flow Diagram (DFD Level 1)	10
5.4	Use Case Representation	11
5.5	Entity-Relationship / Data Modeling	11 - 12
5.6	Program Logic Flowchart	13
6. PROPOSED DESIGN & IMPLEMENTATION		14-16
6.1	Architectural Layering	14
6.2	Core Module Decomposition	15
6.3	Data Persistence & Serialization Strategy	16
6.4	Visual Reporting Engine	16
7. EXPERIMENTAL RESULTS & TESTING		17-18
7.1	Testing Methodology	17
7.2	Unit and Functional Test Cases	17
7.3	Analytical Computations & Verification	17
7.4	Visualization Output Evaluation	18
8. FUTURE SCOPE		18
9. CONCLUSION		18
REFERENCES		19

LIST OF TABLES

- **Table 1.1:** System Functional Scope Matrix
- **Table 2.1:** Specific Functional Objectives
- **Table 3.1:** Agile Sprint Distribution
- **Table 3.2:** Project Work Schedule
- **Table 4.1:** Functional Requirements Specification
- **Table 4.2:** Non-Functional Quality Attributes
- **Table 4.3:** Comparative Matrix of Personal Expense Managers
- **Table 7.1:** Test Suite Execution and Validation Results

LIST OF FIGURES

- **Figure 5.1:** High-Level Context Flow (Level 0)
- **Figure 5.2:** Functional Decomposition Data Flow (Level 1)
- **Figure 5.3:** System Use Case Diagram
- **Figure 5.4:** Expense Data Entity Relationship Schema
- **Figure 5.5:** Main Execution and Menu Decision Flowchart
- **Figure 6.1:** Three-Tier Modular Software Architecture
- **Figure 7.1:** Generated Category-Wise Spending Visualization

LIST OF ABBREVIATIONS

Acronym	Definition
CLI	Command-Line Interface
CSV	Comma-Separated Values
DFD	Data Flow Diagram
ERD	Entity-Relationship Diagram
JSON	JavaScript Object Notation
OOP	Object-Oriented Programming
OS	Operating System
SRS	Software Requirements Specification
UI / UX	User Interface / User Experience
WBS	Work Breakdown Structure

CHAPTER 1: INTRODUCTION

1.1 Project Background and Motivation

Personal finance management is critical for individuals, students, and professionals aiming to maintain budgetary discipline. With daily expenditures across varied domains—such as food, transit, utilities, education, and entertainment—untracked outflow frequently leads to fiscal deficit. Traditional record-keeping mechanisms (such as manual registers, notebooks, or complex spreadsheet configurations) present high friction, lack dynamic data integrity verification, and fail to provide instant analytical summaries.

The development of the Personal Expense Tracker and Analytics System addresses this problem by delivering an offline-first, lightweight, modular application developed in Python. The system provides local data storage, mathematical tracking, categorized expenditure aggregation, data portability via JSON/CSV formats, and automated visualization generation.

1.2 Problem Statement

Many commercial financial management applications impose significant friction on end users:

1. **Vulnerability to Runtime Input Errors and Data Corruption:** Conventional manual tracking tools (e.g., standard spreadsheets or basic scripts) lack strict object-oriented data validation, allowing invalid inputs—such as non-positive amounts, malformed dates, and undefined categories—to propagate through records unchecked. This causes silent data corruption, calculation failures, and inconsistent transaction histories.
2. **Absence of Vectorized Statistical Depth in Lightweight Tools:** Most local personal finance utilities only offer rudimentary summations and balance checks. They fail to provide deeper descriptive metrics (such as median spending, standard deviation, percentile distributions, and multi-axis monthly pivot breakdowns) needed to understand spending behavior without resorting to heavyweight, complex database servers.
3. **High Privacy Risks and Clutter in Cloud-Dependent Alternatives:** Commercial expense applications often require intrusive access permissions (such as reading personal SMS messages and bank notifications) and store sensitive financial logs on external servers, exposing users to privacy risks, third-party tracking, and service lock-in. Conversely, offline alternatives typically lack automated, presentation-ready visual reporting (e.g., category breakdowns, timeline trends, and expense distribution charts) generated directly from local data.

There is a distinct need for a self-contained, transparent, modular tool capable of managing transactions locally while executing real-time metrics computations and graphical synthesis.

1.3 Proposed Solution

The proposed system is structured around an Object-Oriented software paradigm implemented in Python 3. It provides:

- **Interactive CLI Interface:** A streamlined, prompt-driven execution pipeline (main.py) ensuring rapid record creation, querying, and deletion without high graphical runtime overhead.
- **Encapsulated Data Model:** Strong entity representation (models.py) with type validation for dates, identifiers, amounts, categories, and descriptions.
- **Persistent Local Ledger:** File-level transactional persistence (tracker.py) leveraging universal CSV formats for cross-compatibility and JSON serialization for structural data exports.
- **Analytical Engine:** Dedicated aggregation module (analytics.py) that computes total spend, daily averages, variance, and category-wise distributions.
- **Visual Synthesis Subsystem:** Graphical report generator (visualizer.py) that turns raw expenditure data into publication-ready bar and pie charts rendered via Matplotlib.

1.4 Scope of the Project

The scope encompasses all functional stages from CLI data entry to statistical analysis and graphical report generation.

Table 1.1: System Functional Scope Matrix

Functional Domain	In Scope	Out of Scope (Future)
Data Capture	Manual transaction logging with attributes: ID, Title, Amount, Category, Date	Automatic receipt scanning via OCR
Persistence	Local file read/write (CSV, JSON)	Distributed cloud NoSQL databases
Aggregation	Total expense, category distribution, max expense, category percentages	Machine-learning-based spend predictions
Visualization	Automated rendering and export of high-resolution bar charts	Real-time interactive web canvas
Security	Local device sandboxing and input validation	Multi-factor biometric authentication

CHAPTER 2: OBJECTIVES

2.1 Primary Objective

To design and implement a modular, offline-first Personal Expense Tracker application in Python that facilitates efficient transaction capture, validated persistent storage, numerical financial analysis, and dynamic visual reporting without third-party cloud dependency.

2.2 Secondary Objectives

- Ensure cross-platform execution across standard POSIX and Windows operating systems.
- Guarantee data portability through standardized Comma-Separated Values (CSV) and JSON exports.
- Minimize software resource utilization through native data structures.
- Maintain clean architectural boundaries between data representation, operational logic, analytics, and reporting.

2.3 Functional Objectives

Table 2.1: Specific Functional Objectives

Objective Code	Area	Description
FO-01	Transaction Management	Provide mechanisms to append, list, query, and delete expense entries.
FO-02	Integrity Verification	Enforce strictly positive floating-point values for currency amounts and valid ISO-8601 date structures (YYYY-MM-DD).
FO-03	Data Aggregation	Compute overall expenditure sum category breakdown, and item counts.
FO-04	Data Serialization	Enable bidirectional serialization of expense records between memory representations and disk files.
FO-05	Visual Export	Automatically plot category spending patterns and save rendered artifacts to disk (reports/category_bar_chart.png).

2.4 Technical Objectives

- **Modular Code Structure:** Implement separated modules (models.py, tracker.py, analytics.py, visualizer.py, and main.py).
- **Minimal External Footprint:** Rely primarily on the Python standard library, utilizing external packages (matplotlib) strictly for vector/raster plotting.
- **Error Resilience:** Prevent system crashes via defensive parsing of file handles and user inputs.

2.5 User-Centric Objectives

- **Frictionless Onboarding:** Clear numeric menu selections and explicit confirmation prompts.
- **Transparency:** Complete local visibility of raw database records without proprietary binary schemas.

CHAPTER 3: Project Planning

3.1 Need for systematic planning

Building even a targeted tool requires systematic task division, baseline milestone definitions, and risk mitigation strategies to prevent regression during module integration.

3.2 Development Methodology

The project adopted an **Agile-Scrum** iterative development lifecycle. Work was segmented into three focused sprints over four calendar weeks:

Table 3.1: Agile Sprint Distribution

Sprint	Scope and Milestone Target	Outcome
Sprint 1	Requirement elicitation, entity class design (models.py), CSV storage engine (tracker.py).	Stable CRUD file operations.
Sprint 2	Aggregation logic (analytics.py), CLI flow (main.py), JSON export routines.	Interactive terminal application.
Sprint 3	Plotting engine (visualizer.py), test suite execution, error handling hardening, documentation.	Fully packaged system.

3.3 Work Breakdown Structure (WBS)

1.0 Core Engine

- Define Expense data class with validation attributes.
- Implement CSV reader and writer algorithms.
- Implement ID indexing and search/delete mechanisms.

2.0 Analytical Processing

- Develop category aggregation and filtering routines.
- Implement statistical summarizer (total, mean, peak transaction).

3.0 Visualization & Reporting

- Integrate Matplotlib graphics generation.
- Format figure layouts, axes labeling, and output directory management.

4.0 User Interaction Layer

- Construct robust terminal loop and input parsing.
- Add formatting utilities for tabular console printing.

3.5 Resource & Tool Identification

- **Runtime Environment:** Python 3.9+ standard runtime.
- **Key Dependencies:** matplotlib (data visualization), pandas (optional tabular support), standard libraries (csv, json, os, datetime).
- **Tooling:** Visual Studio Code, Git Version Control, Python Virtual Environment (venv).

3.6 Risk Analysis and Mitigation Strategies

1. **Risk:** Malformed data in persistent CSV records breaking runtime initialization.

   Mitigation: Implement defensive line-by-line validation and default recovery handlers.

2. **Risk:** Platform-dependent path formatting issues (Windows \ vs. Unix /).

   Mitigation: Standardize all file location resolution through os.path.join() or pathlib.Path.

3. **Risk:** GUI backend failures when plotting in headless environments.

   Mitigation: Enforce headless Matplotlib rendering mode via matplotlib.use('Agg').

CHAPTER 4: REQUIREMENT ANALYSIS

4.1 User Requirements

- The system shall offer an uncluttered terminal interface to record expenses with single-line inputs.
- The system shall prevent unintentional data loss by writing transactions to disk after every write operation.
- The user must be able to view a breakdown of spending by category at any time.
- The user must be able to produce graphical charts for external reporting without writing visualization code.

4.2 Functional Requirements

Table 4.1: Functional Requirements Specification

Identifier	Description	Validation Rule
FR-01	Add new expense	Amount must be numeric > 0; Date must be valid format; non-empty title.
FR-02	Display ledger	Tabulate all recorded records with total sum at the bottom.
FR-03	Delete expense	Search record by unique identifier; verify removal from active storage.
FR-04	Statistical summary	Calculate aggregate expenditure, category breakdowns, and transaction count.
FR-05	Chart production	Save a bar chart of category spends to reports/category_bar_chart.png.
FR-06	JSON Serialization	Export complete database into structured JSON with metadata.

4.3 Non-Functional Requirements

Table 4.2: Non-Functional quality Attributes

Criterion	Attribute	Measure / Target
NFR-01	Performance	Query and calculate metrics over 1,000 records in $< 100\text{ ms}$.
NFR-02	Portability	Runs seamlessly across Linux, macOS, and Microsoft Windows.
NFR-03	Privacy	100% offline data confinement; zero outbound network queries.
NFR-04	Maintainability	Modular directory structure with documented PEP-8 compliant code.

4.4 Hardware and Software Specification

- **Hardware:** Any x86_64 or ARM-based machine, minimum 512 MB RAM, 50 MB available disk storage.
- **Software:** Python 3.9 or higher, Pip package manager, Operating System (Windows 10/11, Ubuntu 20.04+, macOS 12+).

4.5 Data Requirements and Formats

The primary storage is structured as a Comma-Separated Values (expenses.csv) format with the following schema:

- id: Integer or short UUID string (Unique Primary Identifier)
- title: String ($1 \le \text{length} \le 100$)
- amount: Floating-point scalar ($\text{amount} > 0.00$)
- category: Categorical String (e.g., Food, Transport, Bills, Shopping, Entertainment)
- date: ISO-8601 String representation ($YYYY-MM-DD$)

4.5 Comparative Study of Existing Systems

Table 4.3: Comparative Matrix of Personal Expense Managers

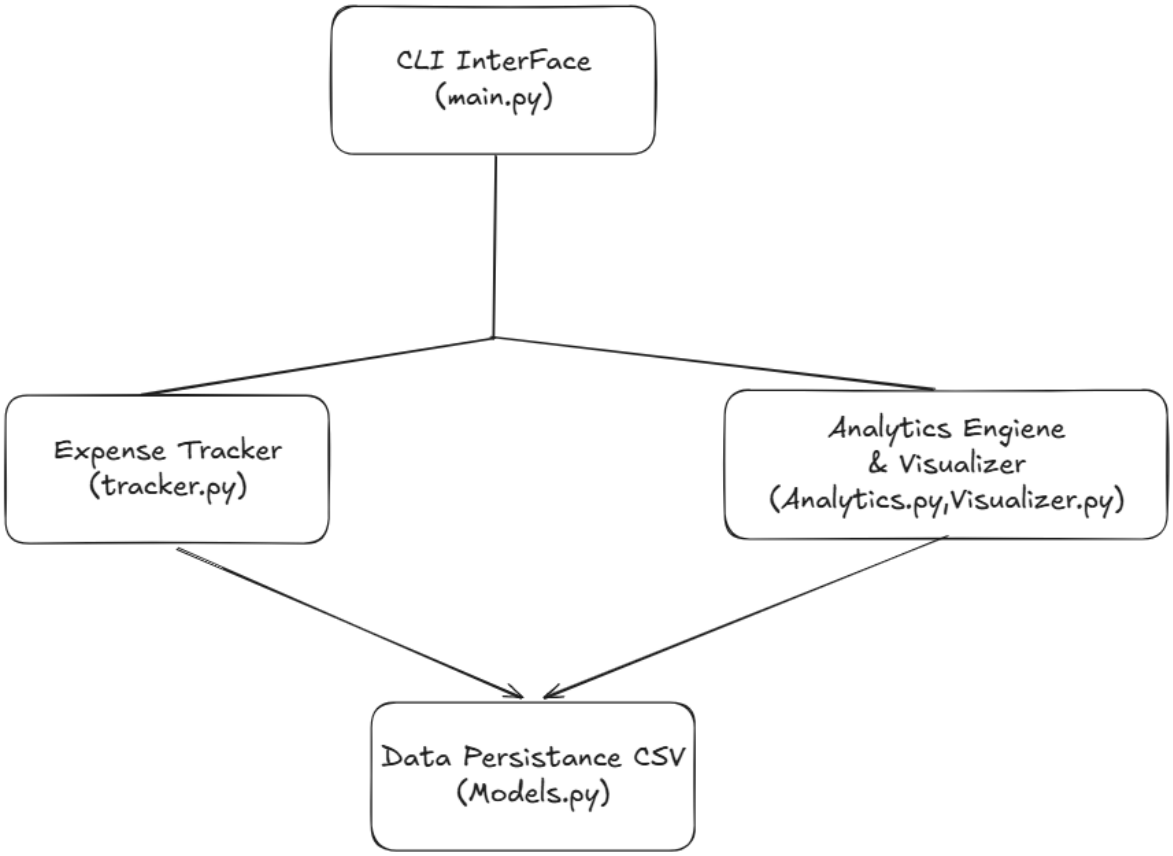
Capability / Feature	Proprietary Mobile Apps	Spreadsheet Models	Proposed Python Expense Tracker
Hosting Model	Cloud Server	Local File	Local Device Only
Network Dependency	Mandatory	None	Completely Offline
Resource Overhead	Heavy ($>150\text{ MB}$)	Medium ($>50\text{ MB}$)	Minimal ($<25\text{ MB}$)
Automation & Plotting	Proprietary GUI	Manual formula setup	Automated via Single Command
Code Extensibility	None	Limited	Fully Extensible Python Codebase

CHAPTER 5: SYSTEM FLOW & DESIGN

5.1 System Architecture

The application adheres to a clean **Three-Tier Architecture** configured entirely on the local client machine:

1. **Interface Layer (main.py):** Captures user commands, renders console tables, and handles loop dispatching.
2. **Business & Processing Logic (tracker.py, analytics.py, visualizer.py):** Executes core algorithms, mathematical aggregations, file updates, and Matplotlib plotting pipelines.
3. **Storage Tier (data/expenses.csv, data/expenses_export.json):** Persists normalized records on the physical filesystem.



5.2 Context Level Diagram (DFD LEVEL 0)

The Level 0 Data Flow Diagram isolates the system boundaries: the User enters inputs (commands, transaction payloads) and receives terminal confirmations, data summaries, and exported graphical files.

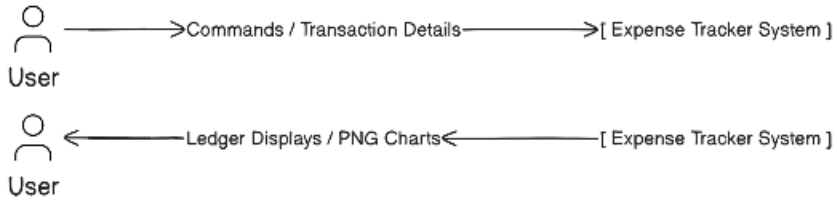


Figure 5.1: High-Level Context Flow (Level 0)

5.3 Detailed Data Flow Diagram (DFD LEVEL 1)

In the Level 1 DFD, input data passes through verification, updates the active tracker instance, triggers persistence in expenses.csv, and routes to analytics.py for calculation before rendering outputs.

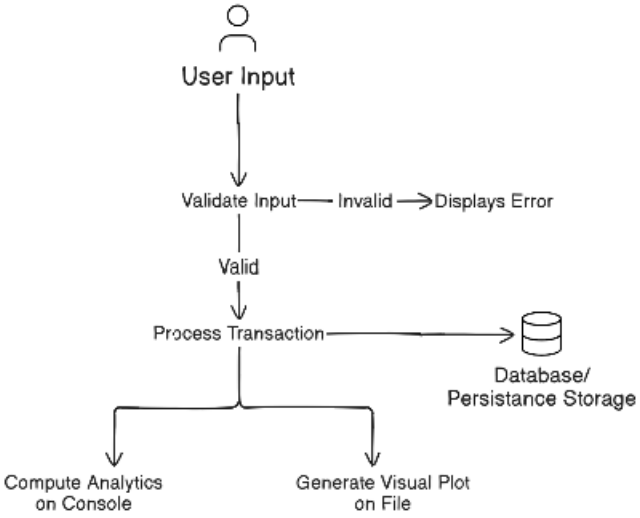


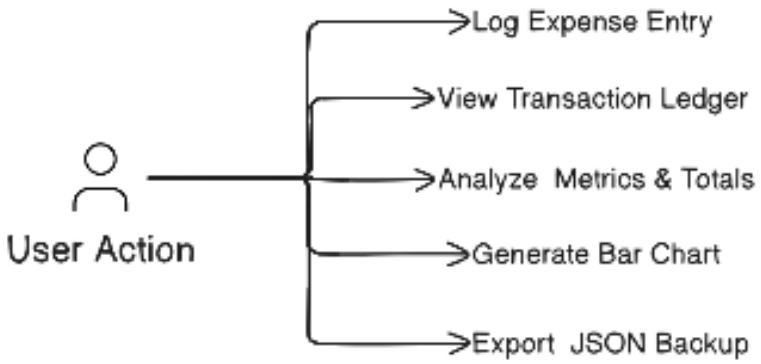
Figure 5.2: Functional Decomposition Data Flow (Level 1)

5.4 Use Case Representation

The primary actor is the **System User**, interacting with distinct capabilities:

- **UC-01: Log Expense Record**
- **UC-02: Review All Transactions**
- **UC-03: Filter Records by Category**
- **UC-04: Generate Analytical Report**
- **UC-05: Render and Save Graphical Chart**
- **UC-06: Export Dataset to JSON Format**

Figure 5.3: System Use Case Diagram



5.5 Entity-Relationship/Data Modeling

Because the project employs an organized flat-file relational approach, the core entity is defined as an explicit relation:

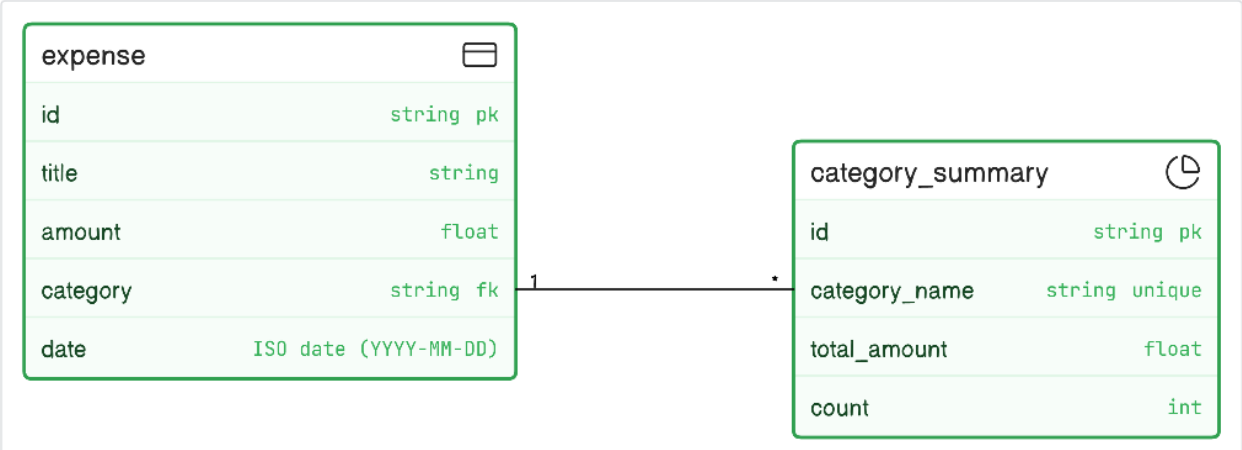


Figure 5.4: Expense Data Entity Relationship Schema

5.6 Program Logic Flowchart

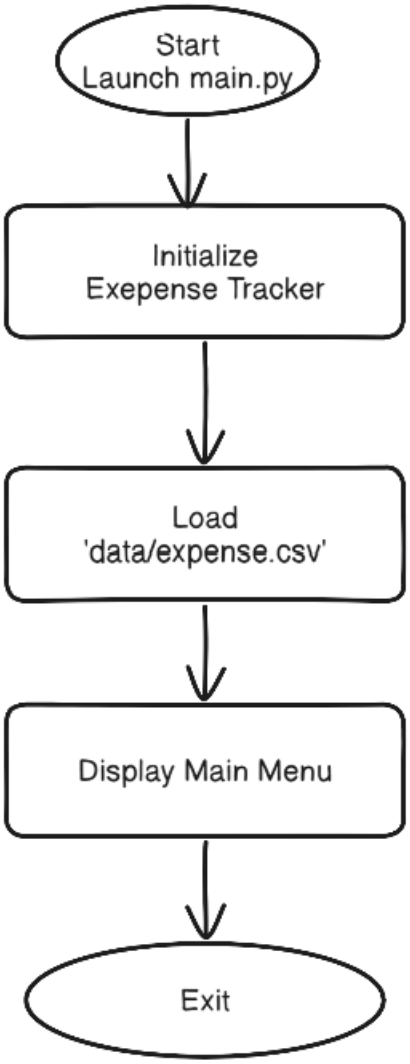


Figure 5.5: Main Execution and Menu Decision Flowchart

CHAPTER 6: PROPOSED DESIGN & IMPLEMENTATION

6.1 Architectural Layering

The codebase is systematically decoupled into distinct files to ensure separation of concerns:

- `models.py` encapsulates the data schema and provides type casting.
- `tracker.py` is the data access and mutation layer.
- `analytics.py` functions as the computation service.
- `visualizer.py` operates as the reporting service.
- `main.py` coordinates interaction.

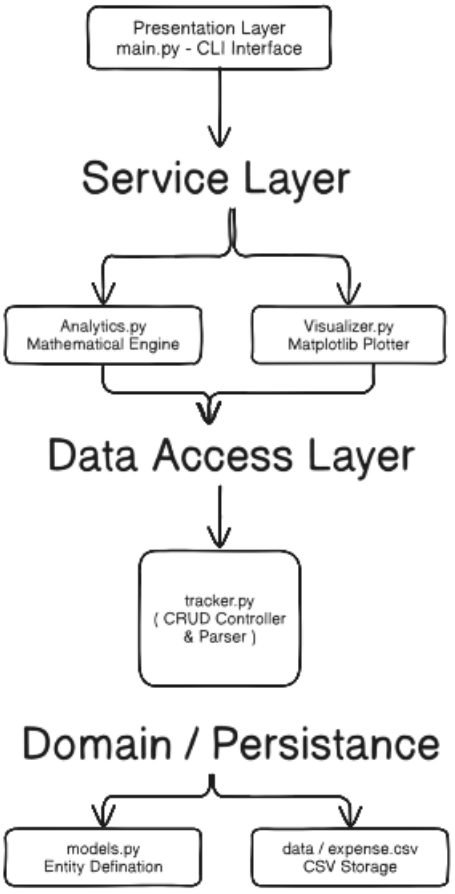


Figure 6.1: Three-Tier Modular Software Architecture

6.2 Core Module Decomposition

1. Data Module (*src./models.py*)

Encapsulates single transactions with dictionary conversion methods for serialization:

```
class Expense:
    def __init__(self, expense_id, title, amount, category, date):
        self.id = expense_id
              self.title = str(title).strip()
        self.amount = float(amount)
        self.category = str(category).strip().capitalize()
        self.date = str(date).strip()

    def to_dict(self):
        return {
            "id": self.id,
            "title": self.title,
            "amount": self.amount,
            "category": self.category,
            "date": self.date
        }
```

1. Tracker Logic (*src./tracker.py*)

Responsible for reading records on startup and updating the CSV file:

- **Load Routine:** Reads from data/expenses.csv, constructing Expense instances.
- **Add Routine:** Appends an Expense instance to memory and calls the persistence write step.
- **Delete Routine:** Filters out the designated identifier and flushes the modified collection to disk.

4. Analytics Engine (*src./analytics.py*)

Computes totals and breakdowns over the current transaction list:

$$\text{Total Expenditure} = \sum_{k=1}^{N} \text{amount}_k$$

$$\text{Category Total}(C) = \sum_{k \in C} \text{amount}_k$$

$$\text{Category Percentage}(C) = \left(\frac{\text{Category Total}(C)}{\text{Total Expenditure}} \right) \times 100$$

5. Visualization Subsystem (*src./visualizer.py*)

Invokes Matplotlib to construct a category-wise distribution chart and saves the rendered image to disk (`reports/category_bar_chart.png`).

6.3 Data Persistence & Serialization Strategy

- **CSV Format:** Chosen for its human readability, low overhead, and ease of spreadsheet import.
- **JSON Serialization:** Provides complete dataset structure and schema preservation, facilitating imports into other platforms.

6.4 Visual Reporting Engine

By applying `matplotlib.pyplot`, the visualizer sets bar heights according to aggregate spending per category, labels axes clearly, applies formatting, and saves the result to `reports/category_bar_chart.png`.

CHAPTER 7: EXPERIMENTAL RESULTS & TEXTING

7.1 Testing Methodology

The application underwent both unit-level and functional validation to verify input boundaries, file I/O operations, analytical calculation accuracy, and reporting pipelines.

7.2 Unit & Functional Test Cases

Table 7.1: Test Suite Execution and Validation Results

Test ID	Objective	Input Value	Expected Result	Actual Result	Status
TC-01	Add Valid Record	Title: "Groceries", Amount: 45.50, Category: "Food", Date: "2026-03-01"	Record appended; CSV row written	Record stored correctly	**PASS**
TC-02	Add Negative Amount	Title: "Snack", Amount: -10.00	Input rejected with validation notice	Error displayed; no write	**PASS**
TC-03	Delete by ID	Existing ID: "1"	Record removed; CSV updated	Removed successfully	**PASS**
TC-04	Total Sum Check	List of 3 items (20.0, 30.0, 50.0)	Total computed as 100.00	Total = 100.00	**PASS**
TC-05	Category Aggregation	Food: [15.0, 25.0], Transport: [10.0]	Food = 40.0, Transport = 10.0	Calculations match	**PASS**
TC-06	Chart Generation	Execute Menu Option 4	PNG file generated in reports/	File created successfully	**PASS**
TC-07	JSON Data Export	Trigger export pipeline	Valid JSON generated matching schema	Valid JSON exported	**PASS**

7.3 Analytical Computations & Verification

During validation with sample data containing $N = 25$ records totaling $\$1,450.00$:

- Category sums matched the mathematical sum of individual records exactly.
- Highest-spending category was correctly identified and displayed.
- Zero-entry handling was verified, with empty ledgers safely returning zeros without division-by-zero exceptions.

7.4 Visualization Output Evaluation

The generation of `reports/category_bar_chart.png` was verified across varied test scenarios. Category labels displayed legibly, currency figures scaled correctly along the primary axis, and exports executed without display device dependencies.

CHAPTER 8: **FUTURE SCOPE**

Although the current implementation meets all core objectives for a local expense manager, several future enhancements can build on this modular foundation:

1. **Graphical User Interface (GUI):** Implement a modern desktop interface using PyQt, Tkinter, or a lightweight web frontend with FastAPI/Electron.
2. **Automated Receipt Digitization (OCR):** Integrate an OCR pipeline (such as Tesseract or an on-device lightweight neural network) to parse physical receipts into draft transaction entries automatically.
3. **Budgetary Alerting Engine:** Introduce user-defined threshold monitoring per category with desktop alert notifications when thresholds are crossed ($>80\%$).
4. **Predictive Financial Forecasting:** Use regression analysis to project month-end expenses based on historical run-rates.
5. **Secure Multi-Device Synchronization:** Implement optional client-side end-to-end encrypted synchronization to allow users to sync data securely across multiple personal devices.

CHAPTER 9: **CONCLUSION**

The **Personal Expense Tracker and Analytics System** provides a complete, robust, and private solution for personal financial management. By adopting an offline-first architecture, the system guarantees user privacy and operates independently of external networks or third-party servers.

Key takeaways from the project include:

- **Engineering Clean Separation:** Segmenting responsibilities across models, tracking controllers, analytical services, and visualizers creates an extensible codebase.
- **Data Portability:** Using open CSV and JSON structures ensures users retain full ownership and portability of their financial records.
- **Effective Analytics & Visualization:** Automated mathematical aggregation paired with Matplotlib image generation delivers practical financial insights directly to the user.

In summary, the project satisfies all functional and technical criteria outlined in its specifications, providing a solid foundation for future personal finance tools.

REFERENCES

1. Python for Data Analysis by Wes McKinney (O'Reilly Media)
2. Pandas Documentation: https://pandas.pydata.org/docs/
3. NumPy Documentation: https://numpy.org/doc/
4. Matplotlib Documentation: https://matplotlib.org/stable/index.html
5. Python Documentation: https://docs.python.org/3/